Práctica 5 RA5 – Gestión de Tienda

Vamos a mejorar el programa de gestión de tienda en base a unos nuevos requerimientos.

Documento funcional

1. Refactor carga inicial inventario en base a una colección usando MongoDB

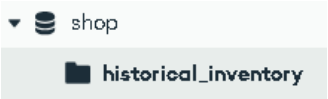
La actual carga inicial de productos al inventario debe leer los datos de una colección `inventory` en base de datos `shop` con el siguiente formato ejemplo.

```
_id: ObjectId('679ed55247c684b34260a366')
name : "Manzana"
▾ wholesalerPrice : Object
    value : 10.5
    currency : "€"
available : true
stock : 10
id : 1

_id: ObjectId('679f3bd911e33cd3a9108455')
name : "Fresa"
▾ wholesalerPrice : Object
    value : 5.5
    currency : "€"
available : true
stock : 20
id : 2
```

2. Refactor exportar datos inventario a colección usando MongoDB

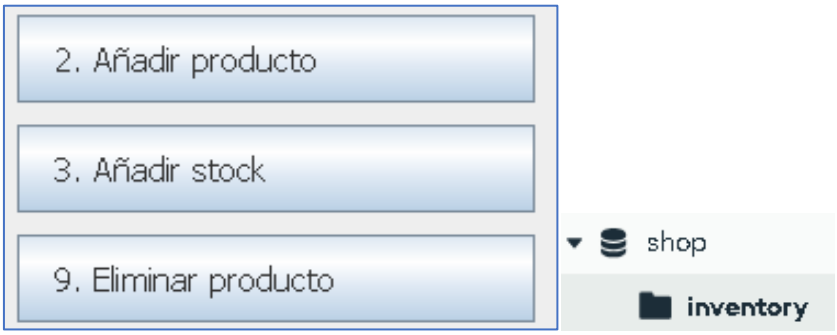
Desde la opción "0. Exportar inventario" en el menú se insertará el actual inventario en una colección de nombre `historical_inventory` en base de datos `shop` con el siguiente formato.

```
_id: ObjectId('679f476029725804639bc4ca')
id : 1
name : "Manzana"
▾ wholesalerPrice : Object
    value : 10.5
    currency : "€"
available : true
stock : 10
created_at : 2025-02-02T10:22:24.924+00:00

_id: ObjectId('679f476029725804639bc4cb')
id : 2
name : "Fresa"
▾ wholesalerPrice : Object
    value : 5.5
    currency : "€"
available : true
stock : 20
created_at : 2025-02-02T10:22:24.934+00:00
```



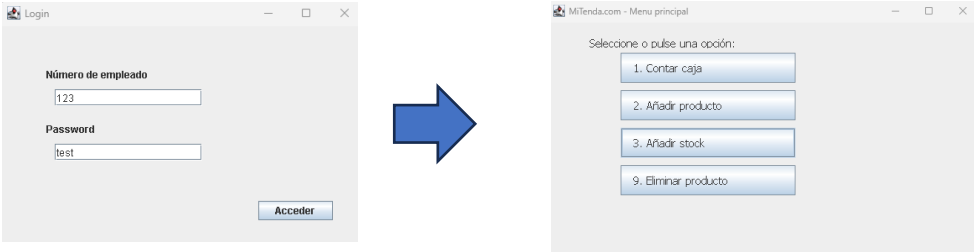
3. Mantenimiento datos inventario en colección usando MongoDB

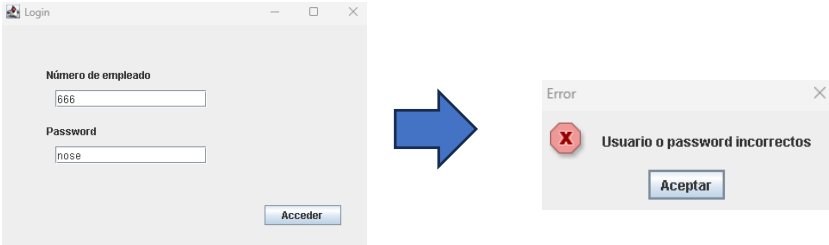
Las opciones que modifiquen datos del producto deben ser reflejadas en la tabla `inventory`



4. Refactor login usando MongoDB

Las credenciales actuales almacenadas en MySQL se sustituirán por almacenados en una base de datos NoSQL en una colección llamad users





Test unitario (Regresión)

1. Verificar carga inventario desde colección `inventory`

2. Verificar exportar inventario a colección
    a. Verificar "Exportar inventario" escribe los datos en colección `historical_inventory` y muestra mensaje informacional
    b. Verificar "Exportar inventario" muestra mensaje error si hubo algún problema al grabar en colección `historical_inventory`.

3. Verificar mantenimiento inventario en colección `inventory`
    a. Verificar "Añadir producto" inserta un nuevo documento
    b. Verificar "Añadir stock" actualiza el stock de un documento ya existente
    c. Verificar "Eliminar producto" elimina un documento ya existente
4. Verificar login correcto accede al menú principal
5. Verificar login incorrecto muestra mensaje de error

Documento Técnico

1. Instalar dependencias mongoDB

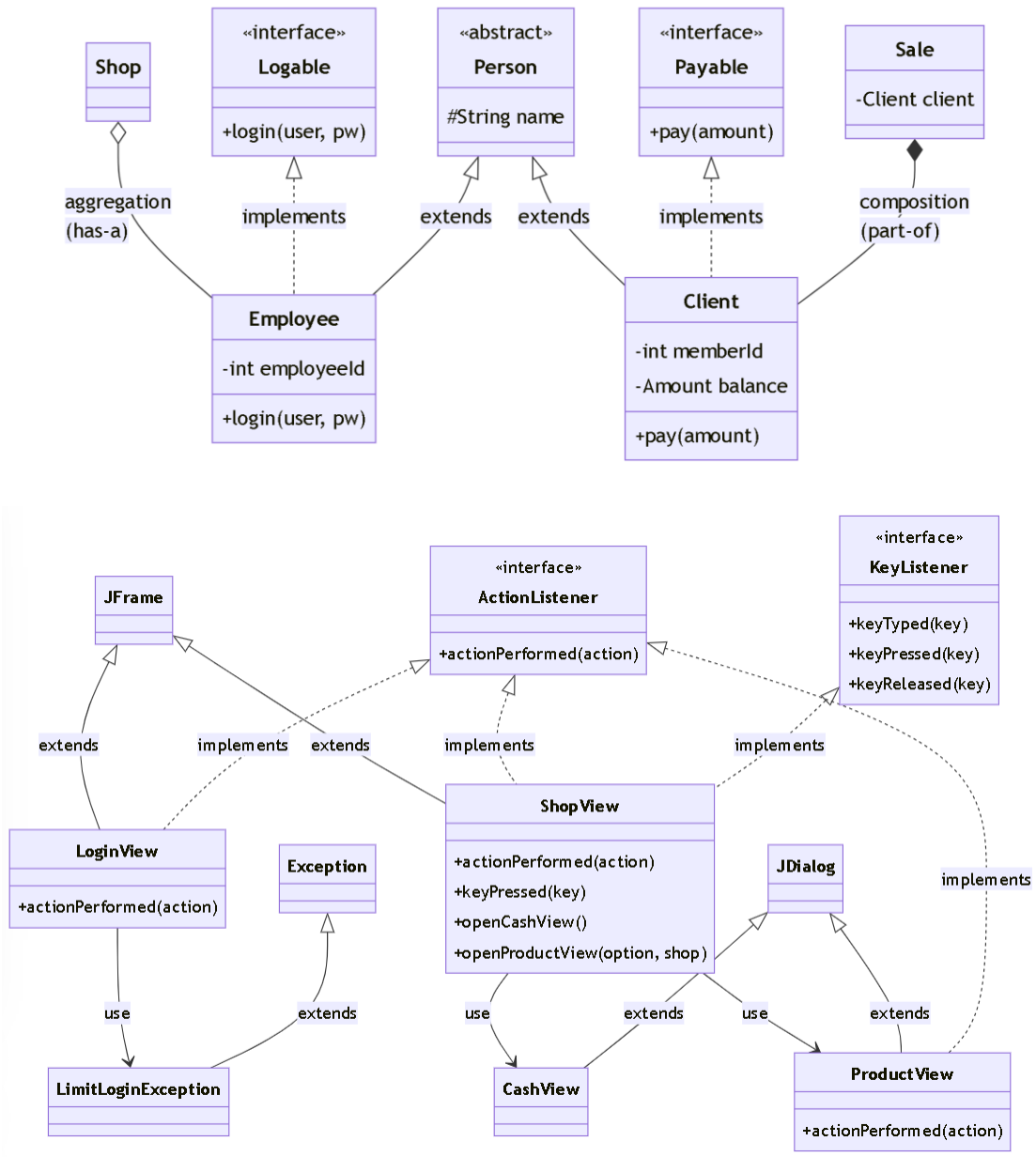
```
<dependency>
        <groupId>org.mongodb</groupId>
        <artifactId>mongo-java-driver</artifactId>
        <version>3.10.2</version>
</dependency>
```

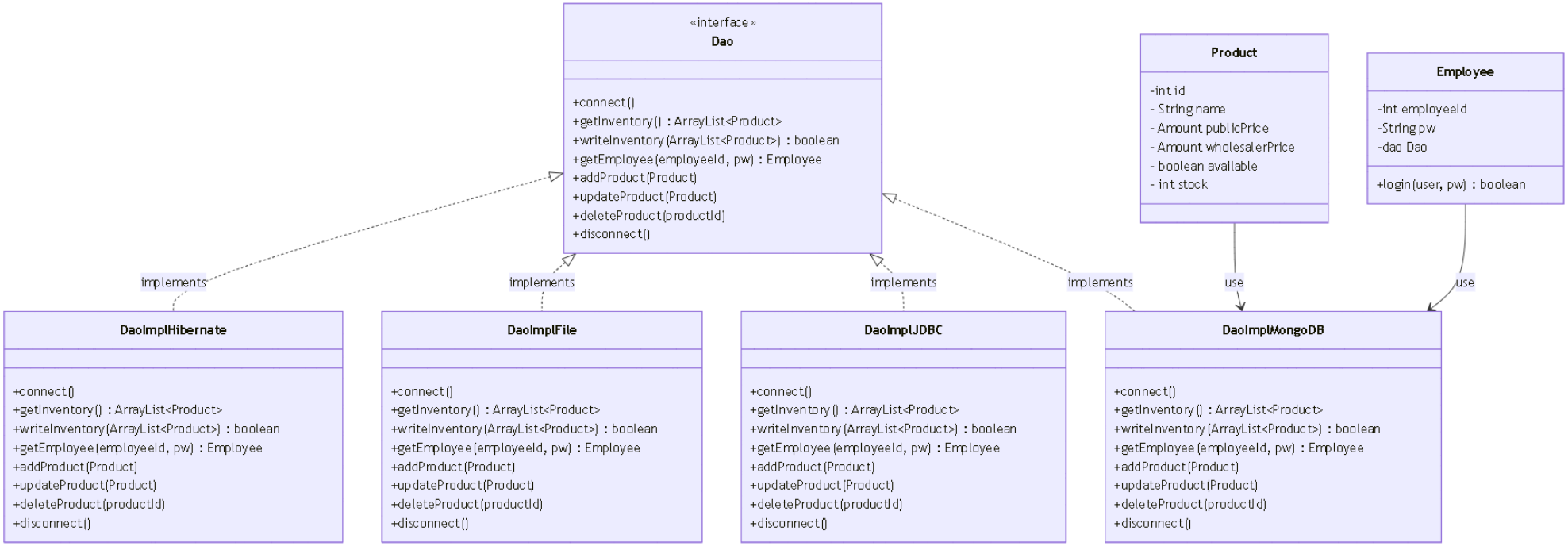
2. Nueva clase DaoImplMongoDB
    a. Crear clase DaoImplMongoDB en package dao
    b. Clase implementa interface Dao
    c. Implementar lógica metodo getInventory
    d. Implementar lógica metodo addProduct
    e. Implementar lógica metodo updateProduct
    f. Implementar lógica metodo removeProduct
    g. Implementar lógica metodo writeInventory
    h. Implementar lógica metodo getEmployee

3. Modificar clase Shop
    a. Modificar tipo atributo shop.dao como DaoImplMongoDB

4. Modificar clase Employee
    a. Modificar tipo atributo employee.dao como DaoImplMongoDB

Diagrama de clases




«interface»
Dao
+connect()
+getInventory() : ArrayList<Product>
+writeInventory(ArrayList<Product>) : boolean
+getEmployee(employeeId, pw) : Employee
+addProduct(Product)
+updateProduct(Product)
+deleteProduct(productId)
+disconnect()
Product
-int id
- String name
- Amount publicPrice
- Amount wholesalerPrice
- boolean available
- int stock
Employee
-int employeeId
-String pw
-dao Dao
+login(user, pw) : boolean
implements
implements
implements
implements
use
use
DaoImplHibernate
+connect()
+getInventory() : ArrayList<Product>
+writeInventory(ArrayList<Product>) : boolean
+getEmployee(employeeId, pw) : Employee
+addProduct(Product)
+updateProduct(Product)
+deleteProduct(productId)
+disconnect()
DaoImplFile
+connect()
+getInventory() : ArrayList<Product>
+writeInventory(ArrayList<Product>) : boolean
+getEmployee(employeeId, pw) : Employee
+addProduct(Product)
+updateProduct(Product)
+deleteProduct(productId)
+disconnect()
DaoImplJDBC
+connect()
+getInventory() : ArrayList<Product>
+writeInventory(ArrayList<Product>) : boolean
+getEmployee(employeeId, pw) : Employee
+addProduct(Product)
+updateProduct(Product)
+deleteProduct(productId)
+disconnect()
DaoImplMongoDB
+connect()
+getInventory() : ArrayList<Product>
+writeInventory(ArrayList<Product>) : boolean
+getEmployee(employeeId, pw) : Employee
+addProduct(Product)
+updateProduct(Product)
+deleteProduct(productId)
+disconnect()